OneDrive tenant-to-tenant migration toolkit.

A PnP PowerShell toolkit for migrating OneDrive for Business content between Microsoft 365 tenants — download-then-upload, manifest-driven, with verified metadata restoration.

What it does.

Four scripts migrate the personal OneDrive (Documents library) of a defined set of users from a source tenant to a target tenant. Files are downloaded to local disk, then uploaded to the target, with a per-file CSV manifest written at each phase and a reconciliation step that proves nothing was lost or truncated. Original Created/Modified timestamps and Created By/Modified By authorship are restored on the target and verified against what the server actually stored.

Content lands in a dedicated subfolder (default: Migrated) inside each target OneDrive, so the migration never collides with files users have already created in a live target account. Only current file versions transfer — version history is skipped by design. Sharing permissions and links do not transfer.

The toolkit suits small-to-medium migrations (single-digit users to low hundreds) where you want full auditability and no third-party tooling. It was proven on a real migration of ~10,000 files / ~32 GB across five users with zero failures and zero reconciliation discrepancies.

Requirements.

- PowerShell 7.4 or later (macOS, Windows, or Linux).
- PnP.PowerShell 2.12 or later (Install-Module PnP.PowerShell).
- SharePoint Administrator rights in both tenants.
- An Entra ID app registration in each tenant (or one multi-tenant app consented in both): public client, redirect URI http://localhost, delegated SharePoint permission AllSites.FullControl with admin consent. PnP.PowerShell no longer ships a shared app registration, so every interactive connect requires a ClientId.
- Local disk space equal to the combined size of all source OneDrives.

```
Register-PnPEntraIDAppForInteractiveLogin -ApplicationName "PnP-Migration" `
    -Tenant <source-tenant>.onmicrosoft.com
Register-PnPEntraIDAppForInteractiveLogin -ApplicationName "PnP-Migration" `
    -Tenant <target-tenant>.onmicrosoft.com
```

Every script accepts -ClientId and falls back to the PNPPOWERSHELL_CLIENTID environment variable; scripts fail fast if neither is present. Tip: if a tenant was created through a self-service or partner flow, its SharePoint hostnames may use the original tenant name (sometimes numeric) rather than the vanity domain — e.g. https://1234567890-my.sharepoint.com. The preflight script resolves and validates the real URLs.

The users.csv control file.

One row per user. All four scripts key off this file; only rows with Include set to Yes are processed.

`DisplayName`	Friendly name used in console output and as the manifest join key across phases.
`SourceUPN`	The user's sign-in name in the source tenant. Also used as the author-map source key.
`SourceOneDriveUrl`	Full personal-site URL in the source tenant, e.g. https://<source>-my.sharepoint.com/personal/<slug>. Validated by preflight.
`TargetUPN`	The user's sign-in name in the target tenant. Used to remap file authorship.
`TargetOneDriveUrl`	Full personal-site URL in the target tenant. Validated by preflight.
`LocalFolder`	Subfolder name under the local staging root for this user's files. Must be unique per user.
`Include`	Yes to migrate this user, No to skip. Lets you stage departed users or pilot one user at a time.

Personal-site slugs follow the pattern of the UPN with dots and @ converted to underscores: first.last@contoso.com becomes /personal/first_last_contoso_com.

Workflow.

```
./0-Preflight.ps1 -SourceClientId <guid> -TargetClientId <guid> -SourceAdminAcct <upn>
./1-Download.ps1  -ClientId <source-guid> -OnlyUser "Pilot User"
./2-Upload.ps1    -ClientId <target-guid> -OnlyUser "Pilot User" -RestoreMetadata
./3-Reconcile.ps1
# pilot clean? run the full pass:
./1-Download.ps1  -ClientId <source-guid>
./2-Upload.ps1    -ClientId <target-guid> -RestoreMetadata
./3-Reconcile.ps1
```

Pilot one user end-to-end first, reconcile clean, then run everyone. A full-batch download plus a full-batch upload produces exactly one manifest pair, which reconcile picks up by default. The first connect of each phase forces fresh authentication so a cached token from the other tenant is never silently reused — sign in with the source-tenant admin for downloads and the target-tenant admin for uploads. Reruns are safe: uploads overwrite and metadata is re-applied.

Parameter reference.

0-Preflight.ps1 — Validate both tenants and grant access.

Resolves each user's real personal-site URL from their user profile on both sides, warns on any mismatch against users.csv, and grants your admin accounts site-collection admin on every in-scope OneDrive. Warns when a target OneDrive has never been provisioned (fix: Request-PnPPersonalSite, or have the user sign in to OneDrive once).

`-UsersCsv`	Path to the control file. Default: users.csv next to the script.
`-SourceAdminUrl`	Source tenant SharePoint admin center URL (https://<tenant>-admin.sharepoint.com).
`-TargetAdminUrl`	Target tenant SharePoint admin center URL.
`-SourceAdminAcct`	Your admin identity in the SOURCE tenant; granted site-collection admin on each source OneDrive. Fails fast if left as a placeholder.
`-AdminAccount`	Your admin identity in the TARGET tenant; granted site-collection admin on each target OneDrive.
`-SourceClientId`	Entra app registration (client) ID for the source tenant. Falls back to PNPPOWERSHELL_CLIENTID.
`-TargetClientId`	Entra app registration (client) ID for the target tenant. Falls back to PNPPOWERSHELL_CLIENTID.

1-Download.ps1 — Source OneDrive to local disk.

Enumerates the Documents library (paged, handles 5,000+ item lists), downloads the current version of every file preserving folder structure, size-verifies each file on disk against the size SharePoint reported, and captures Created, Modified, Author, and Editor per file into the download manifest. A live progress bar shows files done, MB transferred, and the file in flight.

`-ClientId`	Source-tenant app registration ID. Falls back to PNPPOWERSHELL_CLIENTID; required.
`-UsersCsv`	Path to the control file. Default: users.csv next to the script.
`-LocalRoot`	Local staging root. Default: ~/OneDriveMigration. Each user lands in <LocalRoot>/<LocalFolder>.
`-LogDir`	Where manifests are written. Default: logs/ next to the script.
`-OnlyUser`	Process a single user, matched on DisplayName or TargetUPN. Errors if no Include=Yes row matches.

2-Upload.ps1 — Local disk to target OneDrive.

Uploads each user's staged files to the target OneDrive under Documents/<TargetSubfolder>, creating folders as needed and chunking large files automatically. Every local file is gated against the download manifest before upload: files missing from the manifest, recorded as failed, or whose local size drifted are skipped and flagged rather than pushed. Writes an upload manifest including a per-file MetaStatus.

`-ClientId`	Target-tenant app registration ID. Falls back to PNPPOWERSHELL_CLIENTID; required.
`-UsersCsv`	Path to the control file. Default: users.csv next to the script.
`-LocalRoot`	Local staging root the download phase used. Default: ~/OneDriveMigration.
`-LogDir`	Where manifests are read from and written to. Default: logs/ next to the script.
`-OnlyUser`	Process a single user, matched on DisplayName or TargetUPN.
`-TargetSubfolder`	Folder under Documents that receives all migrated content. Default: Migrated. Set to "" to upload into the library root — not recommended against live OneDrives, since uploads overwrite same-named files without warning.
`-RestoreMetadata`	Switch. Restores source Created/Modified timestamps and remaps Created By / Modified By to target users (see Metadata restoration).
`-DownloadManifest`	Explicit path to the download manifest used for gating and metadata. Default: newest download-manifest_*.csv in LogDir.
`-UsersCsvForAuthorMap`	CSV supplying the SourceUPN-to-TargetUPN author mapping. Default: the same users.csv.
`-VerifyEvery`	Metadata verification sampling. Always verifies the first 3 files per user, then every Nth. 1 = verify every file (slowest, maximum assurance); 0 = no verification (unverified files are marked restored-unverified). Default: 25.

3-Reconcile.ps1 — Prove nothing was lost.

Joins the download and upload manifests on user + relative path. Flags files that were downloaded but never successfully uploaded, and files whose uploaded size differs from the downloaded size. Prints a per-user summary (counts, failures, discrepancies, metadata-verified count, MB both sides) and writes a discrepancy CSV when anything is off.

`-LogDir`	Where manifests live. Default: logs/ next to the script.
`-DownloadManifest`	Explicit download manifest path. Default: newest in LogDir.
`-UploadManifest`	Explicit upload manifest path. Default: newest in LogDir. Use explicit paths when reconciling interleaved per-user runs.

Metadata restoration — how and why it works.

With -RestoreMetadata, each uploaded file gets its source Created and Modified timestamps (stored and applied in UTC) plus Created By and Modified By identities written back. Authors are remapped source-to-target via the users.csv UPN columns, resolved in the target tenant with EnsureUser, and passed as claims login-name strings. Identities that cannot resolve (external collaborators, departed users) fall back to the uploading admin for that field; timestamps restore regardless.

Hard-won implementation details.

— User fields are passed as claims login strings, not numeric IDs. Some PnP versions mishandle integer values for Author/Editor and fail with 'The specified user could not be found' (with an empty user name) on every file.

— Author and Editor are always supplied whenever Created/Modified are written. SharePoint validates the user fields during the update; leaving them unset triggers the same empty-user error.

— Updates use UpdateOverwriteVersion, not SystemUpdate. SystemUpdate is known to silently drop the Modified value on document libraries in some PnP/SharePoint Online combinations — the write is accepted, but the server keeps 'now'. UpdateOverwriteVersion persists it. Note it overwrites the current version rather than leaving versioning untouched; on freshly-uploaded migration files there is only v1.0, so nothing is lost — do not repurpose this pattern against documents whose version history matters.

— Every restore is (sampled) verified by reading the item back and comparing the stored Modified against the source value within a 2-second tolerance. An accepted API write is not proof the value survived — this read-back is what catches silent failures. If verification fails, the script waits 5 seconds (to let SharePoint's post-upload processing of Office files settle) and re-applies the dates before flagging the file.

MetaStatus values in the upload manifest.

`restored:<fields>`	Metadata applied and server-verified. Fields list shows exactly what was written.
`restored-unverified:<fields>`	Metadata applied via the same proven path but not individually read back (sampling skipped this file).
`restored-on-retry:<fields>`	First attempt did not stick; the delayed re-apply succeeded and verified.
`restore-verify-FAILED: ...`	Both attempts failed verification; the value the server actually stored is recorded.
`no-source-meta`	The download manifest had no timestamps or authors for this file.

Scope, limits, and gotchas.

— Personal OneDrive Documents library only — not SharePoint team sites, Teams chat files hosted in OneDrive folders, or recycle bins.

— Current versions only; version history does not transfer and target versioning starts fresh.

— Sharing permissions and links do not transfer. Anything shared from the source breaks at cutover and must be re-shared from the target.

— SharePoint Online enforces roughly a 400-character full-URL limit. The download manifest records each path's length and the summary counts anything over 380; remember the target subfolder adds to every

path.

- File names containing #, %, ~, leading/trailing spaces, or emoji can fail either phase; each failure is flagged per file in the manifest rather than aborting the run.
- OneNote notebooks transfer as flat .one/.onetoc2 files but may misbehave in the target; spot-check them after migration.
- Folder timestamps are not restored — only files carry source metadata. SharePoint's activity feed ('created by [admin]') is an immutable audit trail and will always show the uploading admin as the actor; that is history, not file metadata.
- Post-migration hygiene: remove the site-collection admin grants preflight created, revoke or inventory the app registrations (they hold delegated AllSites.FullControl), keep the manifest CSVs as your audit trail, and reclaim local staging disk only after users confirm their content.